ЛАБОРАТОРНАЯ РАБОТА № 2.
Определение и развертывание OLAP-куба

Цель работы – получение навыков создания базы данных, определения и развертывания OLAP-куба с использованием средств языка программирования R.

Задачи

1. Научится устанавливать библиотеки в R и RStudio.
2. Изучить методы создания доступа к хранилищам данных с использованием библиотеки tibble.
3. Изучить методы анализа и обработки данных с использованием библиотеки dplyr.
4. Изучить методы определение и развертывание OLAP-куба с использованием библиотеки rolap.
5. Загрузить данные из файла и развернуть OLAP-куб.
6. Провести анализ агрегированных данных. Выполнить:
   - анализ данных в срезах OLAP-куба;
   - анализ агрегированных данных;
   - анализ динамических данных.
7. Представить результаты анализа в табличном и графическом виде.
8. Самостоятельно выполнить индивидуальное задание.
9. Подготовить отчет по выполненной работе.

2.1. Определение и архитектура системы поддержки принятия решений

К настоящему времени во многих организациях накоплены значительные объемы данных, на основе которых имеется возможность решения разнообразных аналитических и управленческих задач. Проблемы хранения и обработки аналитической информации становятся все более актуальными и привлекают внимание специалистов и фирм, работающих в области

информационных технологий, что привело к формированию полноценного рынка технологий бизнес-анализа.

В идеале работа аналитиков и руководителей различных уровней должна быть организована так, чтобы они могли иметь доступ ко всей интересующей их информации и пользоваться удобными и простыми средствами представления и работы с этой информацией. Именно на достижение этих целей и направлены информационные технологии, объединяющиеся под общим названием хранилищ данных и бизнес-анализа.

В соответствии с определением Gartner, бизнес-анализ (BI, Business Intelligence) – это категория приложений и технологий для сбора, хранения, анализа и публикации данных, позволяющая корпоративным пользователям принимать лучшие решения. В русскоязычной терминологии подобные системы называются также системами поддержки принятия решений (СППР). Архитектура СППР представлена на рис. 2.1.

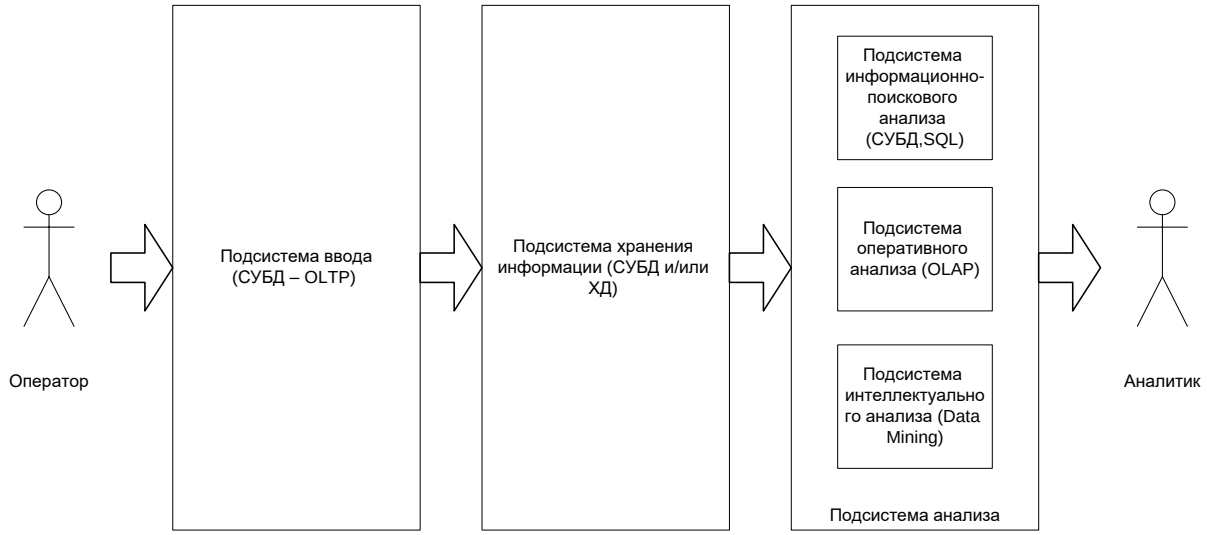


Рисунок 2.1 – Архитектура СППР

Сбор и хранение информации, а также решение задач информационно-поискового запроса эффективно реализуются средствами систем управления базами данных (СУБД). В OLTP (Online Analytical Processing)-подсистемах реализуется транзакционная обработка данных. Непосредственно OLTP-системы не подходят для полноценного анализа информации в силу противоречивости требований, предъявляемых к OLTP-системам и СППР.

Для предоставления необходимой для принятия решений информации обычно приходится собирать данные из нескольких транзакционных баз данных различной структуры и содержания. Основная проблема при этом состоит в несогласованности и противоречивости этих баз-источников, отсутствии единого логического взгляда на корпоративные данные.

Поэтому для объединения в одной системе OLTP и СППР для реализации подсистемы хранения используются концепция хранилищ данных (ХД). В основе концепции ХД лежит идея разделения данных, используемых для оперативной обработки и для решения задач анализа, что позволяет оптимизировать структуры хранения. ХД позволяет интегрировать ранее разъединенные детализированные данные, содержащиеся в исторических архивах, накапливаемых в традиционных OLTP-системах, поступающих из внешних источников, в единую базу данных, осуществляя их предварительное согласование и, возможно, агрегацию.

Подсистема анализа может быть построена на основе:

1. подсистемы информационно-поискового анализа на базе реляционных СУБД и статических запросов с использованием языка SQL;
2. подсистемы оперативного анализа. Для реализации таких подсистем применяется технология оперативной аналитической обработки данных OLAP, использующая концепцию многомерного представления данных;
3. подсистемы интеллектуального анализа, реализующие методы и алгоритмы Data Mining.

2.1.1. Понятие хранилища данных

Технология ХД предназначена для хранения и анализа больших объемов данных с целью дальнейшего обнаружения в них скрытых закономерностей и, наряду с технологией Data Mining, входит в понятие «предсказательная аналитика». Data Mining, в свою очередь, изучает процесс нахождения новых, действительных и потенциально полезных знаний в базах данных.

ХД – предметно-ориентированный, интегрированный, редко меняющийся, поддерживающий хронологию набор данных, организованный для целей поддержки принятия решений. Предметная ориентация означает, что ХД интегрируют информацию, отражающую различные точки зрения на предметную область. Интеграция предполагает, что данные, хранящиеся в ХД, приводятся к единому формату. Поддержка хронологии означает, что все данные в ХД соответствуют последовательным интервалам времени.

Кроме возможности работать с единым источником информации, руководители и аналитики должны иметь удобные средства визуализации данных, агрегирования, поиска тенденций, прогнозирования. Несмотря на многообразие аналитической деятельности можно выделить типовые технологии анализа данных, каждой из которых соответствует определенный набор инструментальных средств. Вместе с хранилищем данных эти средства обеспечивают полное решение для автоматизации аналитической деятельности и создания корпоративной информационно-аналитической системы.

2.1.2. Физические и виртуальные хранилища данных

При загрузке данных из OLTP-системы в ХД происходит дублирование данных. Однако в ходе этой загрузки данные фильтруются, поскольку не все из них имеют значение для проведения процедур анализа. В ХД хранится обобщенная информация, которая в OLTP-системе отсутствует (рис. 2.2).



Рисунок 2.2 – Структура СППР с физическим ХД

Избыточность информации можно свести к нулю, используя виртуальное ХД. В такой системе данные из OLTP-системы не копируются в единое хранилище. Они извлекаются, преобразуются и интегрируются непосредственно при выполнении аналитических запросов в режиме реального времени. Фактически такие запросы напрямую передаются к OLTP-системе.

Достоинства виртуального ХД:

- минимизация объема хранимых данных;
- работа с текущими, актуальными данными.

Недостатки виртуального ХД:

- более высокое, по сравнению с физическим ХД время обработки запросов;
- необходимость постоянной доступности всех OLTP-источников;
- снижение быстродействия OLTP-систем;

OLTP-системы не ориентированы на хранение данных за длительный период времени, по мере необходимости данные выгружаются в архивные, поэтому не всегда имеется физическая возможность получения полного набора данных в ХД.

2.1.3. Проблематика построения хранилищ данных

Основная проблематика при создании ХД заключается в следующем:

1) интеграция разнородных данных. Данные в ХД поступают из разнородных OLTP-систем, которые физически могу быть расположены на различных узлах сети. При проектировании и разработке ХД необходимо решать задачу интеграции различных программных платформ хранения;
2) эффективное хранение и обработка больших объемов данных. Построение ХД предполагает накопление данных за значительные периоды времени, что ведет к постоянному росту объемов дисковой памяти, а также росту объема оперативной памяти, требующейся для обработки этих данных. При возрастании объемов данных этот рост нелинеен;
3) организация многоуровневых справочников метаданных. Конечным пользователям СППР необходимы метаданные, описывающие структуру хранящихся в ХД данных, а также инструменты их визуализации;
4) обеспечение информационной безопасности ХД. Сводная информация о деятельности компании, как правило, относится к коммерческой тайне и подлежит защите; кроме того, в ХД могут содержаться персональные данные клиентов и сотрудников, которые также необходимо защищать. Для выполнения этой функции должна быть разработана политика безопасности ХД и связанной с ним инфраструктуры, а также реализованы предусмотренные в политике организационные и программно-технические мероприятия по защите информации.

2.1.4. Витрины данных

Сокращение затрат на проектирование и разработку ХД может быть достигнуто путем создания витрин данных (ВД). ВД – это упрощенный вариант ХД, содержащий только тематически объединенные данные (рис. 2.3).



Рисунок 2.3 –Структура СППР с самостоятельными ВД

ВД содержит данные, ориентированные на конкретного пользователя, существенно меньше по объему, и для ее реализации требуется меньше затрат. ВД могут строиться как самостоятельно, так и вместе с ХД. ВД внедряются гораздо быстрее и быстрее виден эффект от их использования. Недостатками ВД является многократное хранение одних и тех же данных в различных ВД и отсутствие консолидированности на уровне предметной области.

Обычно информация попадает в ВД из ХД в этом случае ВД называются зависимыми. Возможна также ситуация, когда источником информации для пополнения ВД служат непосредственно OLTP-системы. Такие ВД, получившие название независимых, как правило, рассматриваются как временное решение, позволяющее достаточно быстро и с небольшими затратами решить наиболее важные задачи, оценить преимущества нового подхода, сформулировать некоторые рекомендации для более масштабного проекта разработки общего ХД.

Возможно также совмещение ХД и ВД в рамках одной СППР. ХД в этом случае представляет собой единый источник данных для всей предметной области, а ВД являются подмножествами данных из хранилища, организованными для представления информации по тематическим разделам

данной области. В том случае, если пользователю, для которого создавалась ВД, содержащихся в ней данных недостаточно, то он может обратиться к ХД (рис. 2.4).

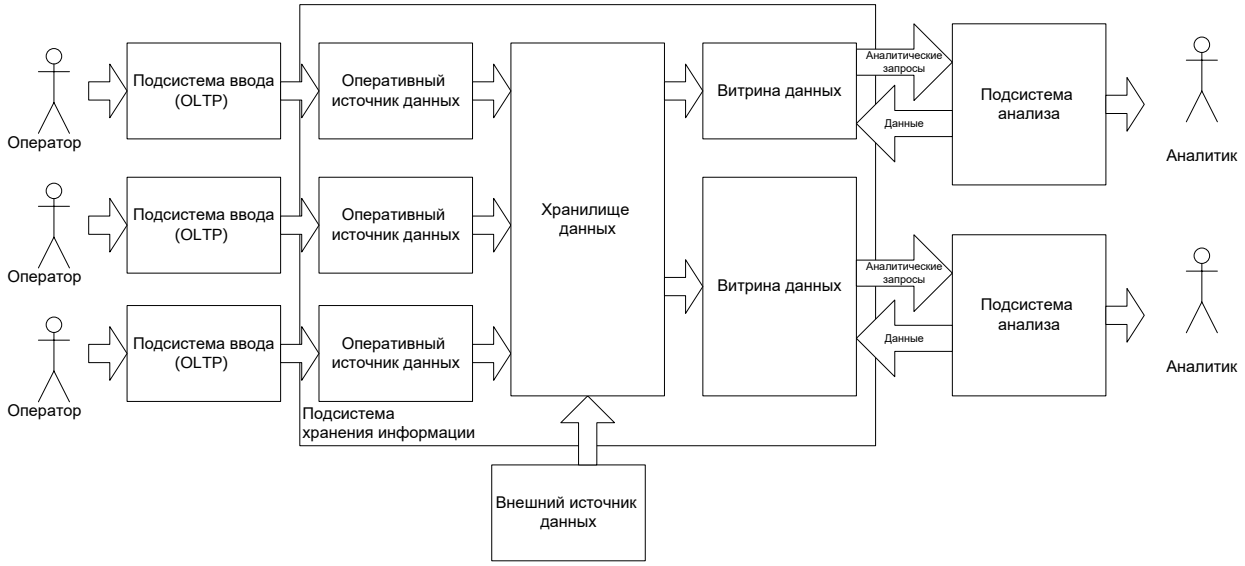


Рисунок 2.4 – Структура СППР с ХД и ВД

Достоинствами такого решения являются простота создания и наполнения ВД, поскольку наполнение происходит из единого стандартизированного источника очищенных данных – из ХД, простота расширения за счет добавления новых ВД, а также снижение нагрузки на основное ХД.

Недостатки заключаются в избыточности, так как данные хранятся и в ХД, и в ВД, а также дополнительные затраты на разработку СППР с ХД и ВД.

2.2. Определение и модель данных OLAP

2.2.1. Понятие OLAP

OLAP (Online Analytical Processing) – технология оперативной аналитической обработки данных, использующая методы и средства для сбора, хранения и анализа многомерных данных в целях поддержки процессов принятия решений.

Основное назначение OLAP-систем – поддержка аналитической деятельности, произвольных запросов пользователей – аналитиков. Цель OLAP-анализа – проверка возникающих гипотез.

2.2.2. Категории данных в ХД

Все данные в ХД делятся на три категории (рис. 2.5).

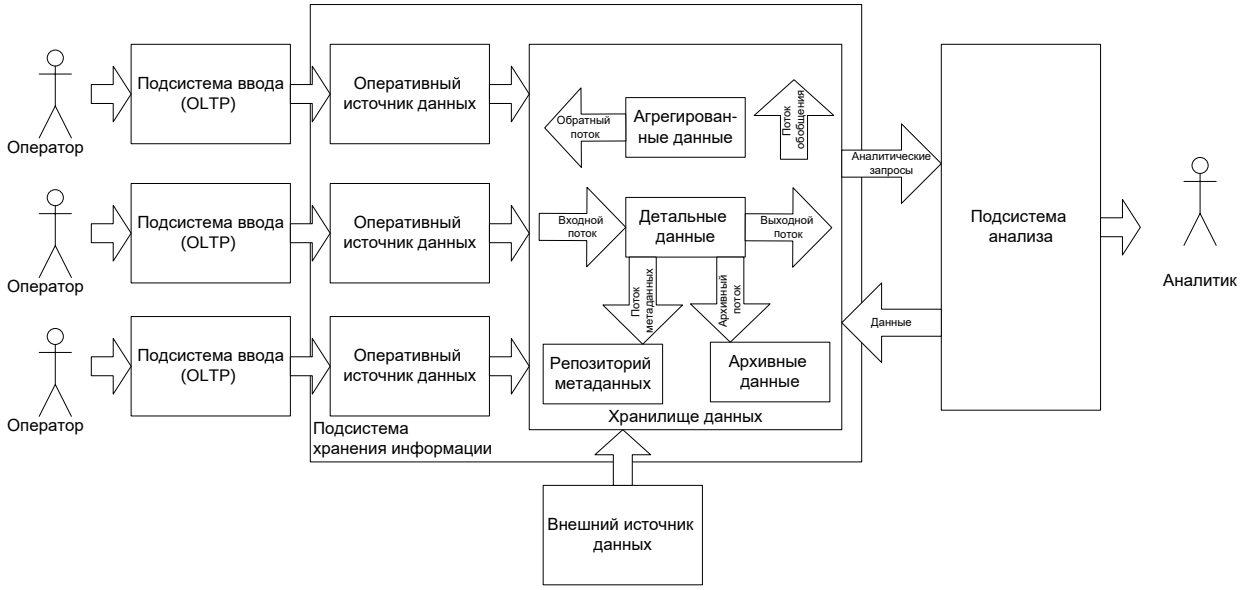


Рисунок 2.5 – Архитектура ХД

Данных, представленные на рис. 2.5, разделены на следующие категории:

1. Детальные данные – данные, переносимые непосредственно из OLTP-подсистем. Соответствуют элементарным событиям, фиксируемым в OLTP-системах. Подразделяются на:

- измерения – наборы данных, необходимые для описания событий (товар, продавец, покупатель, магазин, …);
- факты – данные, отражающие сущность события (количество проданного товара, сумма продаж, …).

2. Агрегированные (обобщенные) данные – данные, получаемые на основании детальных путем суммирования по определенным измерениям.
3. Метаданные – данные о данных, содержащихся в ХД. Могут описывать:

- объекты предметной области, информация о которых содержится в ХД;
- категории пользователей, использующих данные в ХД;

- места и способы хранения данных;
- действия, выполняемые над данными;
- время выполнения различных действий над данными;
- причины выполнения различных действий над данными.

2.2.3. Информационные потоки в ХД

Данные в ХД образуют следующие информационные потоки (см. рис. 2.5):

- входной поток – образуется данными, копируемыми из OLTP-систем в ХД; данные при этом часто очищаются и обогащаются путем добавления новых атрибутов;
- поток обобщения – образуется агрегированием детальных данных и их сохранением в ХД;
- архивный поток – образуется перемещением детальных данных, количество обращений к которым снизилось;
- поток метаданных – образуется потоком информации о данных в репозиторий данных;
- выходной поток – образуется данными, извлекаемыми пользователями;
- обратный поток – образуется очищенными данными, записываемыми обратно в OLTP-системы.

2.2.4. Структура OLAP-куба

В процессе анализа данных часто возникает необходимость построения зависимостей между различными параметрами, число которых может быть значительным.

Под измерением будем понимать последовательность значений одного из анализируемых параметров. Например, для параметра «время» это – последовательность дней, месяцев, кварталов, лет.

Возможность анализа зависимостей между различными параметрами предполагает возможность представления данных в виде многомерной модели – гиперкуба (рис. 2.6).

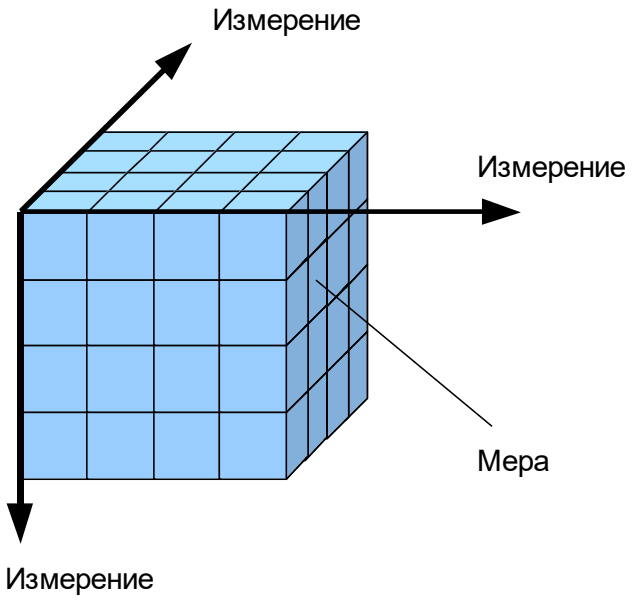


Рисунок 2.6 – Гиперкуб

Оси куба представляют собой измерения, по которым откладывают параметры, относящиеся к анализируемой предметной области, например, названия товаров и названия месяцев года.

На пересечении осей измерений располагаются данные, количественно характеризующие анализируемые факты – меры, например, объемы продаж, выраженные в единицах продукции.

В простейшем случае двумерного куба получается таблица, показывающая значения уровней продаж по товарам и месяцам.

Дальнейшее усложнение модели данных возможно по нескольким направлениям:

1. Увеличение числа измерений – данные о продажах не только по месяцам и товарам, но и по регионам. В этом случае куб становится трехмерным.
2. Усложнение содержимого ячейки – например, нас может интересовать не только уровень продаж, но и чистая прибыль или остаток на складе. В этом случае в ячейке будет несколько значений.

3. Введение иерархии в пределах одного измерения – общее понятие «время» связано с иерархией значений: год состоит из кварталов, квартал из месяцев и т.д.

2.2.5. Иерархия измерений OLAP-кубов

Каждое из измерений OLAP-куба может быть представлено в виде иерархической структуры. Например, измерение «Регион» может иметь следующие уровни иерархии: «страна – федеральный округ – область – город – район».

Некоторые измерения могут иметь несколько уровней иерархического представления, например измерение «время» - представление «год – квартал – месяц - день» и представление «год – неделя – день».

Точно так же в рамках измерения «География» можно ввести уровни «Страна», «Регион», «Область» и «Город».

2.2.6. Операции, выполняемые над гиперкубом

Над гиперкубом могут выполняться следующие операции:

1. Срез (рис. 2.7) – формируется подмножество многомерного массива данных, соответствующее единственному значению одного или нескольких элементов измерений, не входящих в это подмножество.

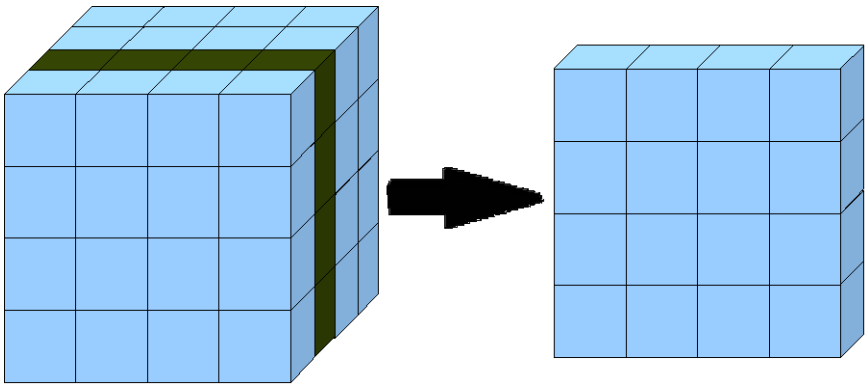


Рисунок 2.7 – Срез

2. Вращение (рис. 2.8) – изменение расположения измерений, представленных в отчете или на отображаемой странице. Например, операция вращения может заключаться в перестановке местами строк и столбцов таблицы. Кроме того, вращением куба данных является

перемещение внетабличных измерений на место измерений, представленных на отображаемой странице, и наоборот.

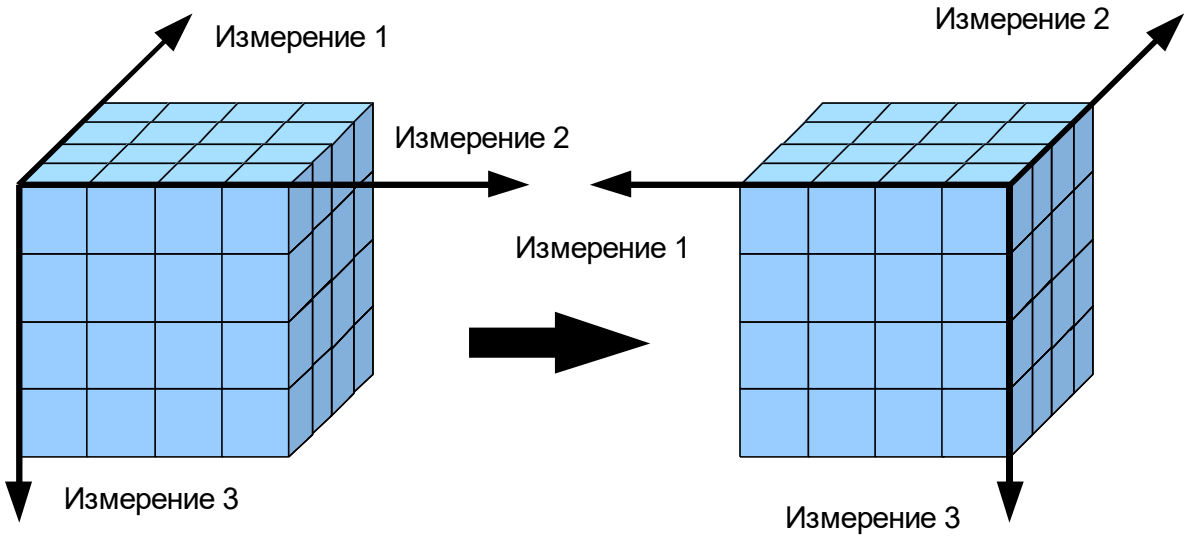


Рисунок 2.8 –Вращение

3. Консолидация (рис. 2.9) и детализация (рис. 2.9) – операции, которые определяют переход вверх по направлению от детального представления данных к агрегированному и наоборот, соответственно. Направление детализации (обобщения) может быть задано как по иерархии отдельных измерений, так и согласно прочим отношениям, установленным в рамках измерений или между измерениями.

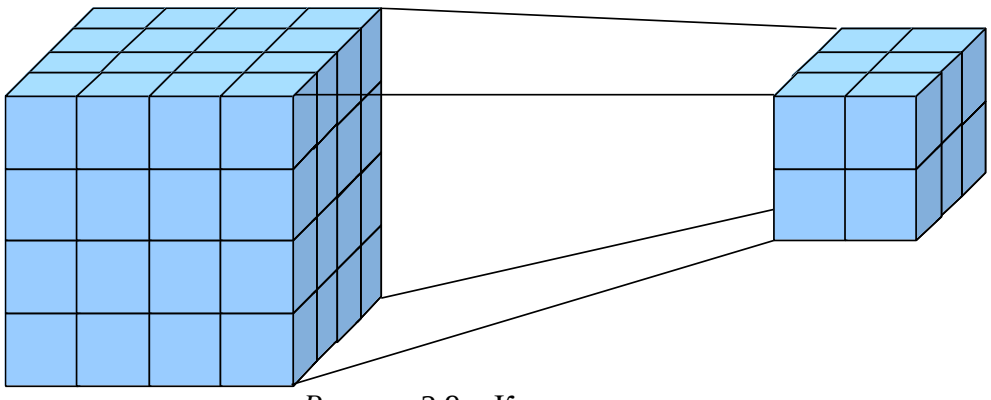


Рисунок 2.9 – Консолидация

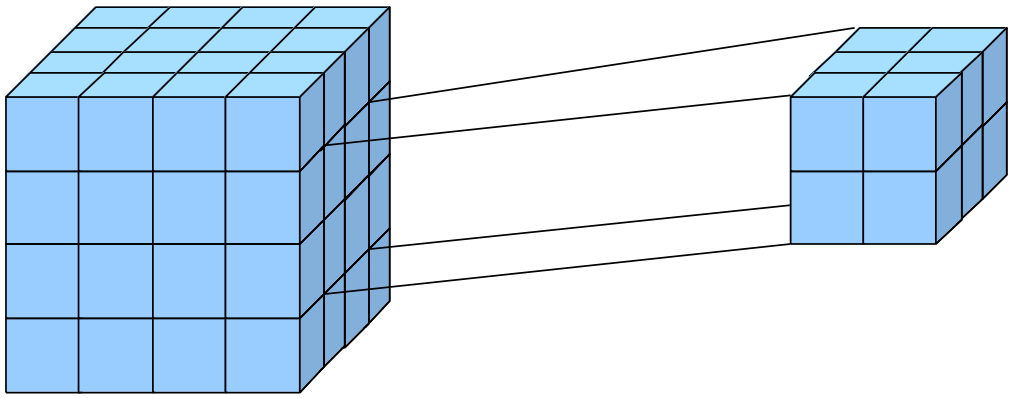


Рисунок 2.10 –Детализация

Например, если при анализе данных о продажах в Северной Америке выполнить операцию детализации для измерения «Регион», то будут отображены такие элементы, как «Канада», «Восточные штаты США» и «Западные штаты США». В результате дальнейшей детализации элемента «Канада» будут отображены элементы «Торонто», «Ванкувер» и т.д.

2.2.7. Таблица фактов

Таблица фактов — является основной таблицей хранилища данных. Как правило, она содержит сведения об объектах или событиях, совокупность которых будет в дальнейшем анализироваться. Обычно говорят о четырех наиболее часто встречающихся типах фактов. К ним относятся:

1. факты, связанные с транзакциями (Transaction facts). Они основаны на отдельных событиях (типичными примерами которых являются телефонный звонок или снятие денег со счета с помощью банкомата);
2. факты, связанные с «моментальными снимками» (Snapshot facts). Основаны на состоянии объекта (например, банковского счета) в определенные моменты времени, например на конец дня или месяца. Типичными примерами таких фактов являются объем продаж за день или дневная выручка;
3. факты, связанные с элементами документа (Line-item facts). Основаны на том или ином документе (например, счете за товар или услуги) и содержат подробную информацию об элементах этого документа (например, количестве, цене, проценте скидки);

4. факты, связанные с событиями или состоянием объекта (Event or state facts). Представляют возникновение события без подробностей о нем (например, просто факт продажи или факт отсутствия таковой без иных подробностей).

Таблица фактов, как правило, содержит уникальный составной ключ, объединяющий первичные ключи таблиц измерений. Чаще всего это целочисленные значения либо значения типа «дата/время» — ведь таблица фактов может содержать сотни тысяч или даже миллионы записей, и хранить в ней повторяющиеся текстовые описания, как правило, невыгодно — лучше поместить их в меньшие по объему таблицы измерений. При этом как ключевые, так и некоторые неключевые поля должны соответствовать будущим измерениям OLAP-куба. Помимо этого, таблица фактов содержит одно или несколько числовых полей, на основании которых в дальнейшем будут получены агрегатные данные.

Для многомерного анализа пригодны таблицы фактов, содержащие как можно более подробные данные (то есть соответствующие членам нижних уровней иерархии соответствующих измерений). В данном случае предпочтительнее взять за основу факты продажи товаров отдельным заказчикам, а не суммы продаж для разных стран — последние все равно будут вычислены OLAP-средством.

В таблице фактов нет никаких сведений о том, как группировать записи при вычислении агрегатных данных. Например, в ней есть идентификаторы продуктов или клиентов, но отсутствует информация о том, к какой категории относится данный продукт или в каком городе находится данный клиент. Эти сведения, в дальнейшем используемые для построения иерархий в измерениях куба, содержатся в таблицах измерений.

2.2.8. Таблицы измерений

Таблицы измерений содержат неизменяемые либо редко изменяемые данные. В подавляющем большинстве случаев эти данные представляют собой

по одной записи для каждого члена нижнего уровня иерархии в измерении. Таблицы измерений также содержат как минимум одно описательное поле (обычно с именем члена измерения) и, как правило, целочисленное ключевое поле (обычно это суррогатный ключ) для однозначной идентификации члена измерения. Если будущее измерение, основанное на данной таблице измерений, содержит иерархию, то таблица измерений также может содержать поля, указывающие на «родителя» данного члена в этой иерархии. Нередко (но не всегда) таблица измерений может содержать и поля, указывающие на «прародителей», и иных «предков» в данной иерархии (это обычно характерно для сбалансированных иерархий), а также дополнительные атрибуты членов измерений, содержавшиеся в исходной оперативной базе данных (например, адреса и телефоны клиентов).

Каждая таблица измерений должна находиться в отношении «один ко многим» с таблицей фактов.

Отметим, что скорость роста таблиц измерений должна быть незначительной по сравнению со скоростью роста таблицы фактов; например, добавление новой записи в таблицу измерений, характеризующую товары, производится только при появлении нового товара, не продававшегося ранее.

2.3. Архитектура OLAP-систем

Полномасштабная OLAP-система должна выполнять сложные и разнообразные функции, включающие сбор данных из различных источников, их согласование, преобразование и загрузку в хранилище, хранение аналитической информации, регламентную отчетность, поддержку произвольных запросов, многомерный анализ и др.

В настоящее время существуют фактические стандарты построения OLAP-систем, основанных на концепции ХД. Эти стандарты опираются на современные исследования и общемировую практику создания хранилищ данных и аналитических систем.

В общем виде архитектура корпоративной OLAP-системы описывается схемой с тремя выделенными слоями (рис. 2.11):

- извлечение, преобразование и загрузка данных;
- хранение данных;
- анализ данных.

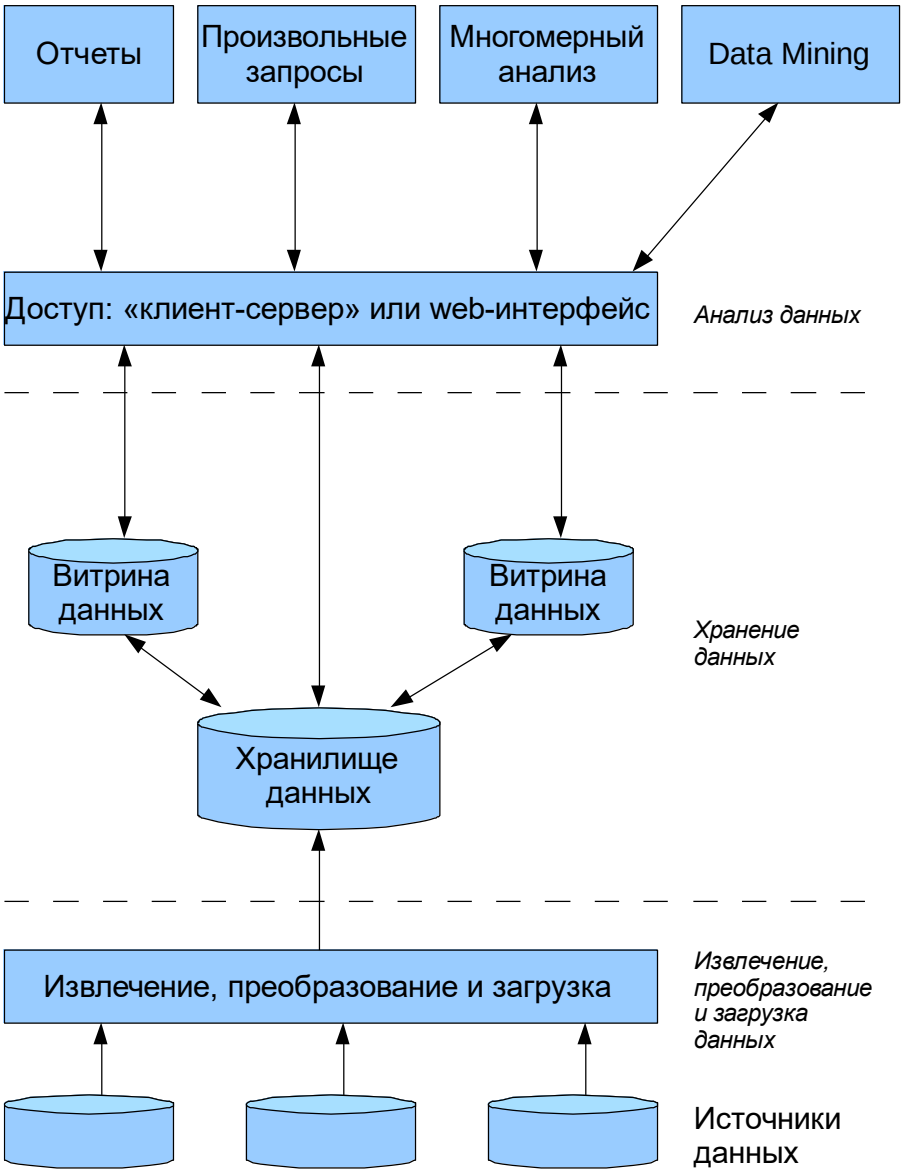


Рисунок 2.11 –Архитектура корпоративной OLAP-системы

Данные поступают из различных внутренних OLTP-систем, от подчиненных структур, от внешних организаций в соответствии с установленным регламентом, формами и макетами отчетности. Вся эта информация проверяется, согласуется, преобразуется и помещается в

хранилище и витрины данных. После этого пользователи с помощью специализированных инструментальных средств получают необходимую им информацию для построения различных табличных и графических представлений, прогнозирования, моделирования и выполнения других аналитических задач.

2.3.1. Слой извлечения, преобразования и загрузки данных

С организационной точки зрения, данный слой включает подразделения и структуры организации всех уровней, поддерживающие базы данных оперативного доступа. Он представляет собой низовой уровень генерации информации, уровень внутренних и внешних информационных источников, вырабатывающих “сырую” информацию. Эта информация является рабочей для повседневной деятельности различных подразделений, которые ее вырабатывают и используют.

С системно-технической точки зрения данный слой представлен ЛВС всех подразделений всех уровней, к которым подключены специализированные технические комплексы, хранящие информацию, чаще всего реализованные в виде реляционных СУБД.

Из источников данных информация перемещается на основе некоторого регламента в централизованное хранилище. Как правило, необходимые для хранилища данные не хранятся в окончательном виде ни в одной из OLTP-систем. Эти данные обычно можно получить из исходных баз данных путем специальных преобразований, вычислений и агрегирования.

Кроме того, несмотря на различную функциональную направленность, исходные транзакционные системы часто «пересекаются» по данным, т.е. их локальные базы данных содержат однотипную по смыслу информацию. Это, прежде всего, касается нормативно-справочной информации, которая используется в том или ином виде в любой OLTP-системе. При этом существенно, что одинаковые по смыслу данные обычно имеют в разных

системах различный формат, вид представления, идентификацию, единицы измерения и т.п. Перед загрузкой в хранилище вся эта информация должна быть согласована, чтобы обеспечить целостность и непротиворечивость аналитических данных.

Согласование данных необходимо и при загрузке данных из одного источника. Дело в том, что в хранилище хранятся исторические данные, т.е. данные за достаточно большой промежуток времени. В оперативной системе данные хранятся в целостном виде за ограниченный промежуток, после чего они отправляются в архив. При изменениях в структуре или собственно данных архивы не подвергаются никакой дополнительной обработке, а хранятся в исходном виде. Следовательно, при необходимости иметь данные за достаточно большой период времени необходимо согласовывать архивную информацию с текущей.

Таким образом, загрузка данных из источников в хранилище осуществляется специальными процедурами, позволяющими:

1. извлекать данные из различных баз данных, текстовых файлов;
2. выполнять различные типы согласования и очистки данных;
3. преобразовывать данные при перемещении их от источников к хранилищу;
4. загружать согласованные и «очищенные» данные в структуры хранилища.

2.3.2. Слой хранения данных

Слой хранения данных предназначен непосредственно для хранения значимой, проверенной, согласованной, непротиворечивой и хронологически целостной информации, которую с достаточно высокой степенью уверенности можно считать достоверной.

Собственно ХД не ориентировано на решение какой-либо определенной функциональной аналитической задачи. Цель ХД – обеспечить целостность и

поддерживать хронологию всевозможных корпоративных данных, и с этой точки зрения оно нейтрально по отношению к приложениям. В связи с этим в большинстве случаев для выполнения определенного комплекса функционально замкнутых аналитических задач рационально создавать витрины данных, в основе которых может быть как многомерная, так и реляционная модель данных. По существу витрина представляет собой относительно небольшое, но что самое важное, функционально-ориентированное ХД, в котором информация хранится специальным образом, оптимизированным с точки зрения решения конкретных аналитических задач некоторого подразделения или группы аналитиков.

ХД чаще всего реализуется в виде реляционной БД, работающей под управлением достаточно мощной реляционной СУБД. Такая СУБД должна поддерживать эффективную работу с терабайтными объемами информации, иметь развитые средства ограничения доступа, обеспечивать повышенный уровень надежности и безопасности, соответствовать необходимым требованиям по восстановлению и архивации.

2.3.3. Слой анализа данных

Для организации доступа аналитиков к данным ХД и ВД используются специализированные рабочие места, поддерживающие необходимые технологии как оперативного, так и долговременного анализа. Результаты работы аналитиков оформляются в виде отчетов, графиков, рекомендаций и сохраняются как на локальном компьютере, так и в общедоступном узле локальной сети.

Аналитическая деятельность в рамках корпорации достаточно разнообразна и определяется характером решаемых задач, организационными особенностями компании, уровнем и степенью подготовленности аналитиков.

В связи с этим современный подход к инструментальным средствам анализа не ограничивается использованием какой-то одной технологи. В

настоящее время принято различать следующие основные вида аналитической деятельности:

5. стандартная отчетность;
6. нерегламентированные запросы;
7. многомерный анализ (OLAP);
8. извлечение знаний (data mining).

Каждая из этих технологий имеет свои особенности, определенный набор типовых задач и должна поддерживаться специализированной инструментальной средой.

2.4. Технические аспекты многомерного хранения данных

OLAP-серверы скрывают от конечного пользователя способ реализации многомерной модели. Они формируют гиперкуб, с которым пользователи посредством OLAP-клиента выполняют необходимые манипуляции, анализируя данные. Однако способ реализации важен, поскольку от него зависят производительность решения и требуемые ресурсы.

Существует три основных способа реализации многомерной модели – MOLAP, ROLAP, HOLAP.

2.4.1. MOLAP

MOLAP (Multidimensional OLAP) – для реализации многомерной модели используются многомерные БД. При этом данные хранятся в виде упорядоченных многомерных массивов. Такие массивы подразделяются на гиперкубы, в которых все хранимые в БД ячейки имеют одинаковую мерность, и поликубы, в которых каждая ячейка хранится с собственным набором измерений. Физически данные хранятся в «плоских» файлах, при этом куб представляется в виде одной плоской таблицы, в которую построчно вписываются все комбинации элементов всех измерений с соответствующими им значениями мер (рис. 2.12).

Измерения				Меры	
Магазин	Время	Поставщик	Товар	Единицы товара	Стоимость товара
№1	01.01.09	Иванов	Картофель	100	20
№1	01.01.09.	Иванов	Морковь	50	25
№1	01.02.09	Иванов	Картофель	150	20
№2	01.02.09	Петров	Морковь	200	25

Рисунок 2.12 – Куб в MOLAP-системе

Преимущества использования многомерных БД в OLAP-системах:

- поиск и выборка данных осуществляется значительно быстрее, чем при многомерном концептуальном взгляде на реляционную БД, так как многомерная БД денормализована и содержит заранее агрегированные показатели, обеспечивая оптимизированный доступ к запрашиваемым ячейкам и не требуя дополнительных преобразований при переходе от множества связанных таблиц к многомерной модели;
- многомерные БД легко справляются с задачами включения в информационную модель разнообразных встроенных функций, тогда как объективно существующие ограничения языка SQL делают выполнение этих задач на основе реляционных БД достаточно сложным, а иногда и невозможным.

Недостатки MOLAP:

- за счет денормализации и предварительно выполненной агрегации объем данных в многомерной БД, как правило, соответствует (по оценке Кодда) в 2,5... 100 раз меньшему объему исходных детализированных данных;
- в подавляющем большинстве случаев информационный гиперкуб является сильно разреженным, а поскольку данные хранятся в упорядоченном виде, в подавляющем большинстве случаев информационный гиперкуб является сильно разреженным, а поскольку данные хранятся в упорядоченном виде, неопределенные значения удается удалить только за счет выбора оптимального порядка сортировки, позволяющего организовать данные в максимально большие непрерывные группы. Кроме того, оптимальный с точки зрения хранения

разреженных данных порядок сортировки, скорее всего, не будет совпадать с порядком, который чаще всего используется в запросах. Поэтому в реальных системах приходится искать компромисс между быстродействием и избыточностью дискового пространства, занятого базой данных;

- многомерные БД чувствительны к изменениям в многомерной модели.

Например, при добавлении нового измерения приходится изменять структуру всей БД, что влечет за собой большие затраты времени.

На основании анализа достоинств и недостатков многомерных БД можно выделить следующие условия, при которых их использование является эффективным:

- объем исходных данных для анализа не слишком велик (не более нескольких гигабайт), т. е. уровень агрегации данных достаточно высок;
- набор информационных измерений стабилен;
- время ответа системы на нерегламентированные запросы является наиболее критичным параметром;
- требуется широкое использование сложных встроенных функций для выполнения кроссмерных вычислений над ячейками гиперкуба, в том числе возможность написания пользовательских функций.

2.4.2. ROLAP

ROLAP (Relational OLAP) – для реализации многомерной модели используются реляционные БД.

В настоящее время распространены две основные схемы реализации многомерного представления данных с помощью реляционных таблиц: схема "звезда" (рис. 2.13) и схема "снежинка" (рис. 2.14).

Если каждое измерение содержится в одной таблице, такая схема хранилища данных носит название «звезда» (star schema). Если же хотя бы одно измерение содержится в нескольких связанных таблицах, такая схема хранилища данных носит название «снежинка» (snowflake schema).

Дополнительные таблицы измерений в такой схеме, обычно соответствующие верхним уровням иерархии измерения и находящиеся в соотношении «один ко многим» в главной таблице измерений, соответствующей нижнему уровню иерархии, иногда называют консольными таблицами (outrigger table).

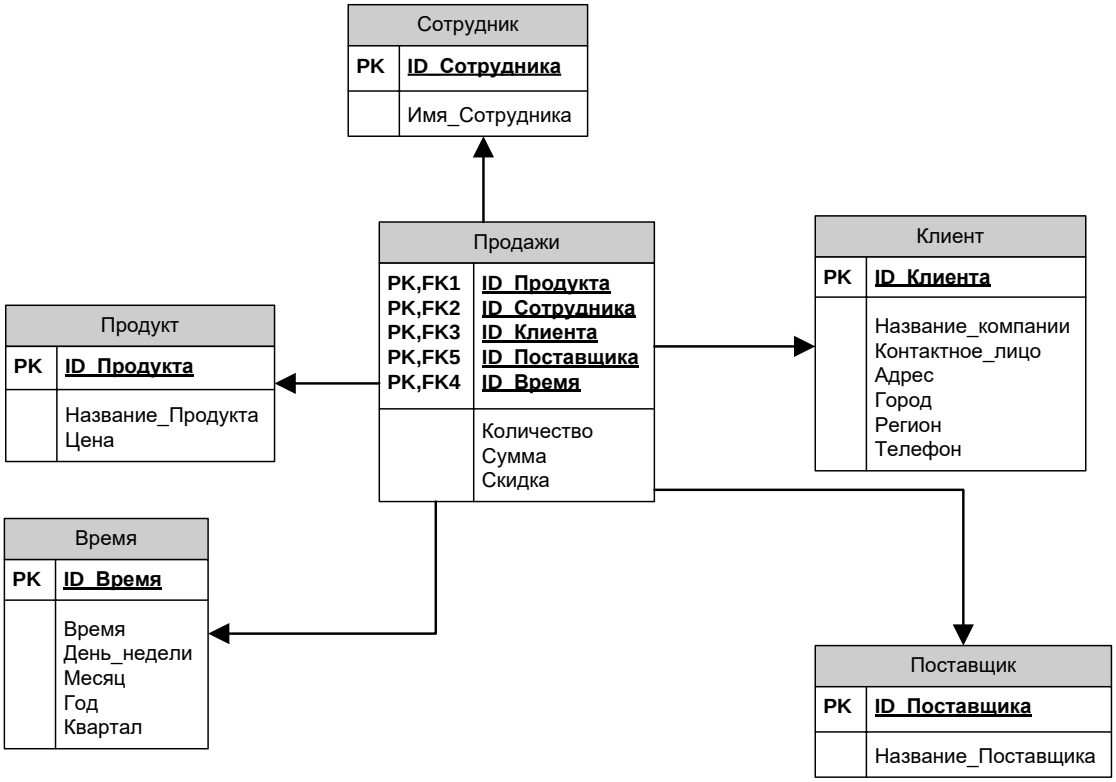


Рисунок 2.13 – Пример схемы данных "звезда"

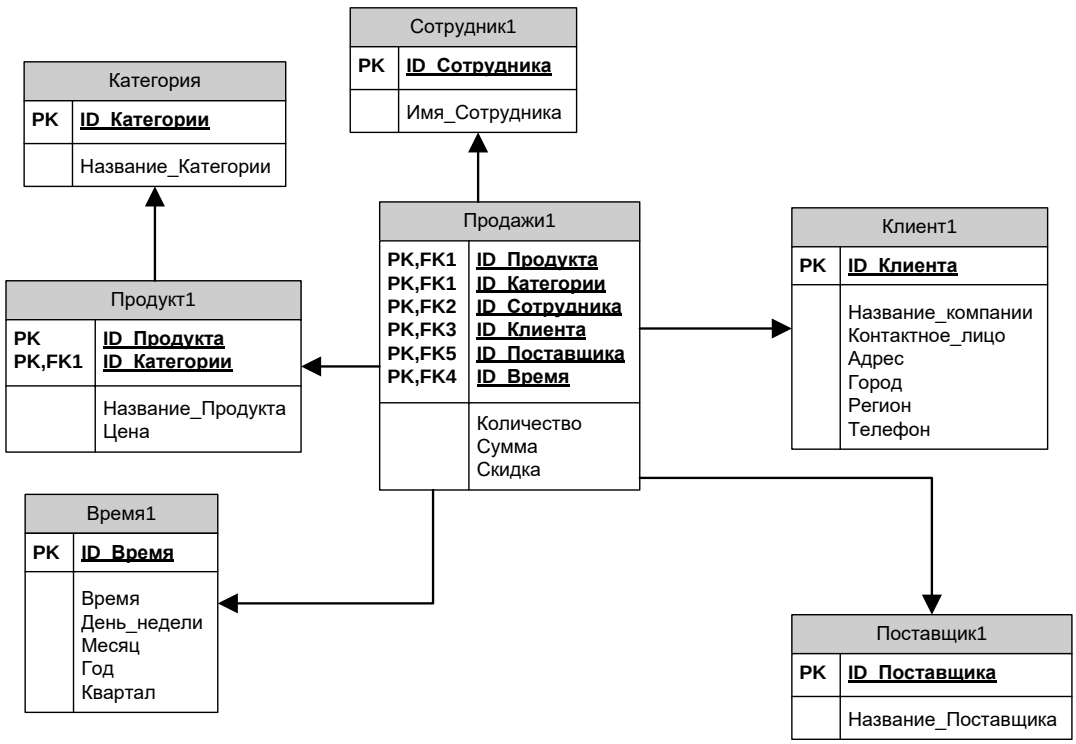


Рисунок 2.14 – Пример схемы данных "снежинка"

В сложных задачах с иерархическими измерениями целесообразно использование схемы "снежинка". В этих случаях отдельные таблицы фактов создаются для возможных сочетаний уровней обобщения различных измерений (рис. 2.14). Это позволяет добиться лучшей производительности, но часто приводит к избыточности данных и к значительным усложнениям в структуре базы данных, в которой оказывается огромное количество таблиц фактов.

Увеличение числа таблиц фактов в БД определяется не только множественностью уровней различных измерений, но и тем обстоятельством, что в общем случае факты имеют разные множества измерений. При абстрагировании от отдельных измерений пользователь должен получать проекцию максимально полного гиперкуба, причем не всегда значения показателей в ней должны являться результатом элементарного суммирования. Таким образом, при большом числе независимых измерений необходимо поддерживать множество таблиц фактов, соответствующих каждому возможному сочетанию выбранных в запросе измерений, что также приводит к неэкономному использованию внешней памяти, увеличению времени загрузки данных в БД со схемой "звезда" из внешних источников и сложностям администрирования.

Использование реляционных БД в OLAP-системах имеет следующие достоинства:

- в большинстве случаев корпоративные ХД реализуются средствами реляционных СУБД, и инструменты ROLAP позволяют производить анализ непосредственно над ними. При этом размер хранилища не является таким критичным параметром, как в случае MOLAP;
- в случае переменной размерности задачи, когда изменения в структуру измерений приходится вносить достаточно часто, ROLAP-системы с динамическим представлением размерности являются оптимальным

решением, т. к. в них такие модификации не требуют физической реорганизации БД;

- реляционные СУБД обеспечивают значительно более высокий уровень защиты данных и хорошие возможности разграничения прав доступа.

Главный недостаток ROLAP по сравнению с многомерными СУБД — меньшая производительность. Для обеспечения производительности, сравнимой с MOLAP, реляционные системы требуют тщательной проработки схемы базы данных и настройки индексов. Только при использовании схем типа "звезда" производительность хорошо настроенных реляционных систем может быть приближена к производительности систем на основе многомерных баз данных.

2.4.3. HOLAP

HOLAP (Hybrid OLAP) - для реализации многомерной модели используются и многомерные, и реляционные БД. HOLAP-серверы используют гибридную архитектуру, которая объединяет технологии ROLAP и MOLAP. В отличие от MOLAP, которая работает лучше, когда данные более-менее плотные, серверы ROLAP показывают лучшие параметры в тех случаях, когда данные довольно разрежены. Серверы HOLAP применяют подход ROLAP для разреженных областей многомерного пространства и подход MOLAP — для плотных областей. Серверы HOLAP разделяют запрос на несколько подзапросов, направляют их к соответствующим фрагментам данных, комбинируют результаты, а затем предоставляют результат пользователю.

2.5. Основы работы с rolap пакетом

Пакет rolap предназначен для выполнения OLAP анализа данных на основе базы данных ROLAP типа “звезда”, состоящие из таблиц фактов и измерений, загрузки данных из таблиц и экспорта результатов обработки данных их в различные форматы для использования инструментами запросов OLAP.

Рассмотрим наглядный пример, демонстрирующий работу пакета на примере набора данных Система отчетности о смертности в 122 городах США за 1962-2016 гг. (табл. 2.1).

Таблица 2.1. Данные по смертности в США

Год	Неделя	Дата окончания недели	Область	Состояние	Город	Смертность от пневмонии и гриппа	Все смерти	<1 года (все причины смерти)
1962	2	13.01.1962	1	И	Бостон	11	270	14
1962	4	27.01.1962	1	И	Бостон	12	285	22
1963	4	26.01.1963	1	И	Бостон	10	276	11
1964	3	18.01.1964	1	И	Бостон	13	325	17
1964	6	02/08/1964	1	И	Бостон	9	244	13
1962	3	20.01.1962	1	КТ	Бриджпорт	2	40	5
1962	5	02/03/1962	1	КТ	Бриджпорт	5	46	6
1962	8	24.02.1962	1	КТ	Бриджпорт	2	45	2
1963	4	26.01.1963	1	КТ	Бриджпорт	2	46	4

В таблице выше показаны все строки набора данных. Для каждой недели и города включены показатели смертности по **возрастной** группе и **причине**, рассматриваемые отдельно (т. е. сочетание возрастной группы и причины не включено). В причине проводится только различие между пневмонией или гриппом и другими. Что касается типов данных, все столбцы имеют тип character, они были импортированы напрямую как tibble, без указания конкретного типа.

Из этого исходного набора данных были определены два дополнительных набора данных ft_num и с помощью преобразований, показанных ниже ft_age.

```
library(rolap)

mrs_ft <-
  read_flat_table_file(name = 'mrs', "mrs_122_us_cities_1962_2016.csv",
                       unknown_value = "Not available")
ft <- mrs_ft |>
  get_table()
ft_num <- ft |>
  dplyr::mutate(`Pneumonia and Influenza Deaths` = as.integer(`Pneumonia and Influenza Deaths`))
|>
  dplyr::mutate(`All Deaths` = as.integer(`All Deaths`))

ft_age <- ft |>
  dplyr::select(-`Pneumonia and Influenza Deaths`,-`All Deaths`) |>
  tidyr::gather("Age", "All Deaths", 7:11) |>
  dplyr::mutate(`All Deaths` = as.integer(`All Deaths`)) |>
  dplyr::mutate(Age = stringr::str_replace(Age, " \\(all cause deaths\\)", ""))
```

Набор данных ft_num имеет ту же структуру и данные, что и ft, только типы данных двух причинных мер изменяются. Для ft_age столбцы с причинными мерами удаляются, и он преобразуется в таблицу (через tidyr::gather), с единственной мерой числового типа и удалением избыточных описаний. ft_num и ft_age являются таблицами, которые будут рассматриваться в качестве отправной точки для получения из них баз данных типа в этом примере.

2.5.1. Пространственное моделирование

Для каждой таблицы цель состоит в том, чтобы определить атрибуты, которые соответствуют мерам в фактах и тем, которые находятся в измерениях. Для фактов должны быть определены меры и их функции агрегации. Для измерений должны быть сгруппированы атрибуты с естественным родством. Каждый атрибут должен появляться в определении только один раз.

Чтобы избежать необходимости писать имя атрибутов таблицы, с помощью следующей функции мы можем иметь их в виде строки. Таким образом, мы можем копировать и вставлять каждое имя по мере необходимости.

```
dput(colnames(ft_num))
#> c("Year", "WEEK", "Week Ending Date", "REGION", "State", "City",
#> "Pneumonia and Influenza Deaths", "All Deaths", "<1 year (all cause deaths)",
#> "1-24 years (all cause deaths)", "25-44 years", "45-64 years (all cause deaths)",
#> "65+ years (all cause deaths)")

dput(colnames(ft_age))
#> c("Year", "WEEK", "Week Ending Date", "REGION", "State", "City",
#> "Age", "All Deaths")
```

2.5.2. Схема «звезда»

Создаем объект star_schema с помощью функции star_schema(), как показано ниже.

```
s <- star_schema()
```

У нас уже есть пустая схема, мы можем добавлять в нее факты и измерения, как показано в следующих разделах.

2.5.3. Размерность

Определим измерения через переменные, используя функцию dimension_schema(), а затем функцию define_dimension() в объекте star_schema, или напрямую в объекте star_schema, используя функцию define_dimension(). В примере ниже результат в обоих случаях одинаков. Преимущество в том, что с помощью переменных мы экономим на определении их несколько раз, если одно и то же измерение появляется в нескольких схемах звезд.

```
# definition 1
when <- dimension_schema(name = "When",
                         attributes = c("Year",
                                        "WEEK"))
s_age <- star_schema() |>
  define_dimension(when)

# definition 2
s_age <- star_schema() |>
  define_dimension(name = "When",
                         attributes = c("Year",
                                        "WEEK"))
```

2.5.4. Факты

Для полноты факты также могут быть определены как переменные или напрямую связаны с объектом star_schema. Что касается измерений, в примере ниже результат в обоих случаях одинаков.

```
# definition 1
mrs_cause <- fact_schema(name = "MRS Cause",
                measures = c("Pneumonia and Influenza Deaths",
                             "All Deaths"))
s_cause <- star_schema() |>
  define_facts(mrs_cause)

# definition 2
s_cause <- star_schema() |>
  define_facts(name = "MRS Cause",
                measures = c("Pneumonia and Influenza Deaths",
                             "All Deaths"))
```

С каждой мерой связана функция агрегации, по умолчанию это *SUM*. В качестве альтернативы можно использовать функции *MAX*, *MIN* или *AVG* использовать нельзя, поскольку среднее значение средних значений не является общим средним значением. Вместо этого nrow_agg всегда добавляется поле с именем (можно указать альтернативное имя), в котором хранится количество записей, добавленных в строку результата. С помощью этого значения и результата *SUM* можно получить значение функции *AVG* .

Функции агрегации необходимы для получения данных с гранулярностью, определенной в схеме «звезда», которая может быть равна или грубее гранулярности, определенной в таблице.

```
s_cause <- star_schema() |>
  define_facts(name = "MRS Cause",
               measures = c("Pneumonia and Influenza Deaths",
                            "All Deaths"),
               agg_functions = c("MAX", "SUM"),
               nrow_agg = "Num"
               )
```

В приведенном выше примере функции агрегации были явно определены для мер, а имя меры по умолчанию, соответствующее количеству добавленных записей, было изменено.

2.5.5. Базы данных звезд

Из объекта star_schema и таблицы (tibble), содержащей все поля, которые в ней встречаются, с помощью функции star_database() получаем объект star_database. В качестве примера рассмотрим данные о причинах.

Определение схемы «звезда» для данных, рассматриваемых в случае причины, показано ниже.

```
s_cause <- star_schema() |>
  define_facts(name = "MRS Cause",
               measures = c("Pneumonia and Influenza Deaths",
                            "All Deaths"),
               nrow_agg = "Num") |>
  define_dimension(name = "When",
                           attributes = c("Year",
                                          "WEEK",
                                          "Week Ending Date")) |>
  define_dimension(name = "Where",
                           attributes = c("REGION",
                                          "State",
                                          "City"))
```

Далее мы получаем базу данных звезд.

```
db_cause <- star_database(s_cause, ft_num)
```

На данный момент целью этой базы данных является экспорт ее таблиц для использования в инструментах запросов OLAP. Для этой цели предлагаются две функции: во-первых, snake_case() преобразование имен столбцов в Snake Case, а во-вторых, as_tibble_list() генерация tibble списка с таблицами базы данных, как показано в следующем примере.

```
l_cause <- db_cause |>
  snake_case() |>
  as_tibble_list()
```

Видно, что данные в таблице фактов имеют ту же степень детализации, что и данные в исходной таблице (они имеют одинаковое количество записей).

```
nrow(ft_num)
#> [1] 20

nrow(l_cause[[3]])
#> [1] 20
```

То же самое можно увидеть в num поле в таблице фактов, которое показывает количество записей, добавленных для получения каждой записи, в данном случае это всегда 1. Измерения содержат только отдельные комбинации значений для наборов атрибутов, которые они содержат.

2.5.6. Переименовывание меры фактов и атрибуты измерений

В данном случае это не было сочтено необходимым, но при желании можно переименовать как атрибуты измерений, так и меры фактов.

Для этого доступны функции get_attribute_names() и для просмотра текущих имен, а также для их изменения при необходимости.

2.5.7. Созвездия

На основе двух или более баз данных звезд мы можем определить созвездие.

2.5.8. Определение звездных баз данных

Чтобы более эффективно продемонстрировать функциональность, мы определим звездные базы данных с более грубой детализацией, чем исходные таблицы; кроме того, мы выберем только некоторые строки из таблиц.

Ниже показан случай причины.

```
when <- dimension_schema(name = "When",
                         attributes = c("Year"))
```

```
where <- dimension_schema(name = "Where",
                          attributes = c("State",
                                         "City"))
s_cause <- star_schema() |>
  define_facts(name = "MRS Cause",
               measures = c("Pneumonia and Influenza Deaths",
                            "All Deaths")) |>
  define_dimension(when) |>
  define_dimension(where)

ft_num2 <- ft_num |>
  dplyr::filter(Year > "1962") |>
  dplyr::filter(City == "Boston" | City == "Bridgeport")

db_cause <- star_database(s_cause, ft_num2) |>
  snake_case()

l_cause <- db_cause |>
  as_tibble_list()
```

Далее мы определяем звездную базу данных и остальные операции в случае возрастных групп.

```
s_age <- star_schema() |>
  define_facts(name = "MRS Age",
               measures = c("All Deaths")) |>
  define_dimension(when) |>
  define_dimension(where) |>
  define_dimension(name = "Who",
                          attributes = c("Age"))

ft_age2 <- ft_age |>
  dplyr::filter(Year < "1964") |>
  dplyr::filter(City != "Boston" & City != "Bridgeport")

db_age <- star_database(s_age, ft_age2) |>
  snake_case()

l_age <- db_age |>
  as_tibble_list()
```

После фильтрации данных можно увидеть, что таблицы измерений содержат только необходимые данные для каждой звездной базы данных, но для общих измерений они имеют одинаковую структуру.

2.5.9. Определение созвездия

Далее мы определяем созвездие, образованное двумя базами данных звезд.

```
ct <- constellation("MRS", db_cause, db_age)
```

Таблицы созвездий также можно экспортировать в виде списка tibble.

```
lc <- ct |>
  as_tibble_list()
```

Ниже приведены таблицы баз данных звезд созвездия. Экземпляры измерений были интегрированы, так что таблицы являются общими для обеих баз данных.

Эти таблицы можно напрямую экспортировать в формат, требуемый инструментом запросов OLAP, который мы собираемся использовать.

Созвездия могут быть определены из баз данных звезд, а также из других созвездий. Например, давайте определим новую базу данных звезд на основе предыдущих примеров.

```
s_age2 <- star_schema() |>
  define_facts(name = "MRS Age 2",
               measures = c("All Deaths")) |>
  define_dimension(when) |>
  define_dimension(where) |>
  define_dimension(name = "Who",
                           attributes = c("Age"))

db_age2 <- star_database(s_age2, ft_age) |>
  snake_case()
```

Далее мы определяем созвездие, оно состоит из ранее определенного созвездия и новой базы данных звезд.

```
ct2 <- constellation("MRS2", ct, db_age2)
```

2.5.10. Экспорт результатов

Для продолжения работы с результатом реализации многомерной базы данных (базы данных звезд или созвездий) ее можно экспортировать в виде списка tibble, пригодного, например, для передачи данных в электронную таблицу, или в виде объекта класса dm, позволяющего передавать данные в реляционные СУБД.

В некоторых случаях нам может быть интересно снова иметь данные в виде таблицы, особенно если мы хотим провести статистический анализ с интегрированными данными. Мы можем экспортировать их как единый список tibble.

2.5.11. Список tibble

Ниже показаны результаты, полученные для базы данных типа «звезда».

```
db_cause |>
  as_tibble_list()
#> $when
#> # A tibble: 2 × 2
#>   when_key year
#>      <int> <chr>
#> 1        1 1963
#> 2        2 1964
#>
#> $where
#> # A tibble: 2 × 3
#>   where_key state city
#>       <int> <chr> <chr>
```

```
#> 1         1 CT    Bridgeport
#> 2         2 MA    Boston
#>
#> $mrs_cause
#> # A tibble: 4 × 5
#>   when_key where_key pneumonia_and_influenza_deaths all_deaths nrow_agg
#>      <int>     <int>                          <int>      <int>    <int>
#> 1        1         1                              2         46        1
#> 2        1         2                             10        276        1
#> 3        2         1                              8         45        1
#> 4        2         2                             22        569        2
```

Также для созвездия.

```
ct |>
  as_tibble_list()
#> $when
#> # A tibble: 3 × 2
#>   when_key year
#>      <int> <chr>
#> 1        1 1962
#> 2        2 1963
#> 3        3 1964
#>
#> $where
#> # A tibble: 4 × 3
#>   where_key state city
#>       <int> <chr> <chr>
#> 1         1 CT    Bridgeport
#> 2         2 CT    Hartford
#> 3         3 MA    Boston
#> 4         4 MA    Cambridge
#>
#> $who
#> # A tibble: 5 × 2
#>   who_key age
#>     <int> <chr>
#> 1       1 1-24 years
#> 2       2 25-44 years
#> 3       3 45-64 years
#> 4       4 65+ years
#> 5       5 <1 year
#>
#> $mrs_cause
#> # A tibble: 4 × 5
#>   when_key where_key pneumonia_and_influenza_deaths all_deaths nrow_agg
#>      <int>     <int>                          <int>      <int>    <int>
#> 1        2         1                              2         46        1
#> 2        2         3                             10        276        1
#> 3        3         1                              8         45        1
#> 4        3         3                             22        569        2
#>
#> $mrs_age
#> # A tibble: 15 × 5
#>    when_key where_key who_key all_deaths nrow_agg
#>       <int>     <int>   <int>      <int>    <int>
#>  1        1         2       1          2        2
#>  2        1         2       2          3        2
#>  3        1         2       3         35        2
#>  4        1         2       4         54        2
#>  5        1         2       5         10        2
#>  6        1         4       1          0        1
#>  7        1         4       2          2        1
#>  8        1         4       3          7        1
#>  9        1         4       4         29        1
#> 10        1         4       5          1        1
#> 11        2         2       1          5        3
#> 12        2         2       2         13        3
#> 13        2         2       3         51        3
```

```
#> 14        2         2       4        107        3
#> 15        2         2       5         16        3
```

2.5.12. Объект dm

Пакет dm позволяет связывать таблицы, реализованные с использованием tibble класса в R, с таблицами в СУБД.

В следующем примере показано преобразование базы данных «звезда» в объект класса dm.

```
# star database
db_cause_dm <- db_cause |>
  as_dm_class()
class(db_cause_dm)
#> [1] "dm"
db_cause_dm
#> ── Metadata ─────────────────────────────────────────────────────────────────────
#> Tables: `when`, `where`, `mrs_cause`
#> Columns: 10
#> Primary keys: 3
#> Foreign keys: 2
```

Также для созвездия.

```
# constellation
ct_dm <- ct |>
  as_dm_class(pk_facts = TRUE)
class(ct_dm)
#> [1] "dm"
ct_dm
#> ── Metadata ─────────────────────────────────────────────────────────────────────
#> Tables: `when`, `where`, `who`, `mrs_cause`, `mrs_age`
#> Columns: 17
#> Primary keys: 5
#> Foreign keys: 5
```

Например, объект dm можно использовать для представления таблиц созвездий, как показано ниже (его также можно использовать для их хранения в СУБД).

```
ct_dm |>
  dm::dm_draw(rankdir = "LR", view_type = "all")
```

В приведенном ниже примере локальная база данных *SQLite* используется для демонстрации того, как сохранить модель в базе данных. dmработает с любой СУБД, поддерживаемой DBI.

```
db <- DBI::dbConnect(RSQLite::SQLite())
ct_dm_db <- dm::copy_dm_to(db, ct_dm)
ct_dm_db
#> ── Table source ─────────────────────────────────────────────────────────────────
#> src:  sqlite 3.44.2 []
#> ── Metadata ─────────────────────────────────────────────────────────────────────
#> Tables: `when`, `where`, `who`, `mrs_cause`, `mrs_age`
#> Columns: 17
#> Primary keys: 5
#> Foreign keys: 5
DBI::dbDisconnect(db)
```

2.5.13. Единый список tibble

Ниже показаны результаты, полученные для базы данных типа «звезда», рассматриваемой как один список tibble. Каждый из них является результатом выполнения *внутреннего соединения* между таблицей фактов и таблицами измерений.

```
db_cause |>
  as_single_tibble_list()
#> $mrs_cause
#> # A tibble: 4 × 6
#>   year  state city       pneumonia_and_influenza_deaths all_deaths nrow_agg
#>   <chr> <chr> <chr>                             <int>      <int>    <int>
#> 1 1963  CT    Bridgeport                            2         46        1
#> 2 1963  MA    Boston                               10        276        1
#> 3 1964  CT    Bridgeport                            8         45        1
#> 4 1964  MA    Boston                               22        569        2
```

Также для созвездия.

```
ct |>
  as_single_tibble_list()
#> $mrs_cause
#> # A tibble: 4 × 6
#>   year  state city       pneumonia_and_influenza_deaths all_deaths nrow_agg
#>   <chr> <chr> <chr>                             <int>      <int>    <int>
#> 1 1963  CT    Bridgeport                            2         46        1
#> 2 1963  MA    Boston                               10        276        1
#> 3 1964  CT    Bridgeport                            8         45        1
#> 4 1964  MA    Boston                               22        569        2
#>
#> $mrs_age
#> # A tibble: 15 × 6
#>    year  state city      age         all_deaths nrow_agg
#>    <chr> <chr> <chr>     <chr>            <int>    <int>
#>  1 1962  CT    Hartford  1-24 years           2        2
#>  2 1962  CT    Hartford  25-44 years          3        2
#>  3 1962  CT    Hartford  45-64 years         35        2
#>  4 1962  CT    Hartford  65+ years           54        2
#>  5 1962  CT    Hartford  <1 year             10        2
#>  6 1962  MA    Cambridge 1-24 years           0        1
#>  7 1962  MA    Cambridge 25-44 years          2        1
#>  8 1962  MA    Cambridge 45-64 years          7        1
#>  9 1962  MA    Cambridge 65+ years           29        1
#> 10 1962  MA    Cambridge <1 year              1        1
#> 11 1963  CT    Hartford  1-24 years           5        3
#> 12 1963  CT    Hartford  25-44 years         13        3
#> 13 1963  CT    Hartford  45-64 years         51        3
#> 14 1963  CT    Hartford  65+ years          107        3
#> 15 1963  CT    Hartford  <1 year             16        3
```

Пакет rolap предлагает набор операций, которые позволяют нам преобразовывать таблицы в звездные базы данных. Из нескольких звездных баз данных можно определить созвездие с согласованными размерами.

Операции были разработаны так, чтобы быть интуитивно понятными и простыми в использовании. Результат значительно облегчает процесс преобразования данных для раскрытой ситуации.

Полученную реализацию многомерной базы данных можно экспортировать для работы с инструментами многомерного анализа в электронных таблицах или СУБД.

2.6. Задание на лабораторную работу

1. Подключиться в https://www.kaggle.com/ и скачать данные в формате *.csv в соответствии со своим вариантом (номер варианта соответствует номеру по списку в учебной группе).
2. Загрузить и описать данные в формате *.csv.
3. Развернуть OLAP-куб и сделать логические выводы по результатам анализа данных.
4. Подготовить отчёт по выполненной работе в формате Word/PDF.

2.7. Варианты заданий

1. Latest Global Temperatures (https://www.kaggle.com/datasets/csafrit2/latest-global-temperatures)
2. Finance companies in India (https://www.kaggle.com/datasets/anuragbantu/finance-companies-in-india)
3. Black Friday Sales EDA(https://www.kaggle.com/datasets/pranavuikey/black-friday-sales-eda)
4. Football Transfers from 1992/93 to 2021/22 seasons(https://www.kaggle.com/datasets/cbhavik/football-transfers-from-199293-to-202122-seasons)
5. World, Region, Country GDP/GDP per capita (https://www.kaggle.com/datasets/tmishinev/world-country-gdp-19602021)
6. Air Pollution Level (https://www.kaggle.com/datasets/totoro29/air-pollution-level)
7. Stack Overflow Developer Survey 2011-2022 (https://www.kaggle.com/datasets/yasirabdaali/stack-overflow-developer-survey-20112022)
8. Financing Healthcare (https://www.kaggle.com/datasets/programmerrdai/financing-healthcare)
9. Law School Admissions Bar Passage (https://www.kaggle.com/datasets/danofer/law-school-admissions-bar-passage)
10. Vending Machine Sales (https://www.kaggle.com/datasets/awesomeasingh/vending-machine-sales)
11. Global Companies dataset (https://www.kaggle.com/datasets/narayan63/global-

companies-dataset)

12. Smoke Detection Dataset (https://www.kaggle.com/datasets/deepcontractor/smoke-detection-dataset)
13. Netflix Data: Cleaning, Analysis and Visualization (https://www.kaggle.com/datasets/ariyoomotade/netflix-data-cleaning-analysis-and-visualization)
14. World Population by Countries Dataset (1960-2021) (https://www.kaggle.com/datasets/kaggleashwin/population-dataset)
15. Online Retails Sale Dataset (https://www.kaggle.com/datasets/rohitmahulkar/online-retails-sale-dataset)
16. Spotify unpopular songs (https://www.kaggle.com/datasets/estienneggx/spotify-unpopular-songs)
17. Chocolate Bar Ratings (https://www.kaggle.com/datasets/evangower/chocolate-bar-ratings)
18. Airbnb Open Data (https://www.kaggle.com/datasets/arianazmoudeh/airbnbopendata)
19. IMDB Movies (https://www.kaggle.com/datasets/totoro29/imdb-movies)
20. Youtube Statistics (https://www.kaggle.com/datasets/advaypatil/youtube-statistics)

Список литературы

1. Полубояров В.В. Использование MS SQL Server Analysis Services 2008 для построения хранилищ данных
2. rolap: Obtaining Star Databases from Flat Tables. – URL: https://CRAN.R-project.org/package=rolap
3. dbplyr: A 'dplyr' Back End for Databases– URL: https://CRAN.R-project.org/package=dbplyr
4. tibble: Simple Data Frames – URL: https://CRAN.R-project.org/package=tibble
5. Многомерное моделирование (учебник по Adventure Works)– URL: https://learn.microsoft.com/ru-ru/analysis-services/multidimensional-models-olap-logical-cube-objects/partitions-partition-storage-modes-and-processing?view=asallproducts-allversions
6. olapR (пакет R в Службах машинного обучения SQL Server) https://learn.microsoft.com/ru-ru/sql/machine-learning/r/ref-r-olapr?view=sql-server-ver16